

Code Explanation

Delta Live Tables (DLT) Pipeline Code Explanation

=====

This is a Delta Live Tables (DLT) pipeline code written in Python, utilizing Apache Spark for data processing. The pipeline reads customer and order data from CSV files, cleans and transforms the data, joins the datasets, and finally aggregates customer spend.

Overview

The code defines a DLT pipeline that ingests data from CSV files, applies data quality checks, and performs various data transformations to produce a set of curated tables. The pipeline consists of multiple stages, including data ingestion, data cleaning, data joining, and data aggregation.

Step 1: Define Catalog and Schema

The code starts by defining the catalog and schema names, which are used to store the processed data in Databricks.

```
CATALOG = "gen_ai_poc_databrickscoe"
SCHEMA = "sdlc_wizard"
```

Step 2: Specify Source Data Paths

The code specifies the paths to the source customer and order data CSV files.

```
CUSTOMER_PATH = "/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/customerdata"
ORDER_PATH = "/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/orderdata"
```

Step 3: Read Source Customer and Order Data

The code uses the `@dlt.table` decorator to define two tables, ``customer_bronze`` and ``order_bronze``, which read the customer and order data from the specified CSV files.

```
@dlt.table(
    name="customer_bronze",
    comment="Raw customer data from CSV files"
)
def customer_bronze():
    return spark.read.option("header", "true").csv(CUSTOMER_PATH)

@dlt.table(
    name="order_bronze",
    comment="Raw order data from CSV files"
)
def order_bronze():
    return spark.read.option("header", "true").csv(ORDER_PATH)
```

Step 4: Clean Customer and Order Data

The code defines two tables, `customer` and `order`, which clean the customer and order data by removing nulls and duplicates.

```
@dlt.table(
    name="customer",
    comment="Cleaned customer data with nulls and duplicates removed"
)
def customer_silver():
    return dlt.read("customer_bronze").dropna().dropDuplicates()

@dlt.table(
    name="order",
    comment="Cleaned order data with TotalAmount calculated and nulls/duplicates removed"
)
def order_silver():
    return (
        dlt.read("order_bronze")
        .dropna()
        .dropDuplicates()
        .withColumn("PricePerUnit", col("PricePerUnit").cast("double"))
        .withColumn("Qty", col("Qty").cast("integer"))
        .withColumn("TotalAmount", col("PricePerUnit") * col("Qty"))
    )
```

Step 5: Join Customer and Order Data

The code defines a table, `customer\_order\_joined`, which joins the cleaned customer and order data on the `CustId` column.

```
@dlt.table(
    name="customer_order_joined",
    comment="Joined customer and order data"
)
def customer_order_joined():
    customer_df = dlt.read("customer")
    order_df = dlt.read("order")

    return customer_df.join(order_df, "CustId", "inner").select(
        "CustId", "Name", "EmailId", "Region", "OrderId", "ItemName",
        "PricePerUnit", "Qty", "Date", "TotalAmount"
    )
```

Step 6: Implement SCD Type 2 for Ordersummary

The code defines a table, `ordersummary`, which implements Slowly Changing Dimension (SCD) Type 2 logic to track changes to the customer order data over time.

```

@dlt.table(
    name="ordersummary",
    comment="SCD Type 2 implementation of customer order data",
    table_properties={
        "delta.enableChangeDataFeed": "true",
        "delta.autoOptimize.optimizeWrite": "true",
        "delta.autoOptimize.autoCompact": "true"
    }
)
@dlt.expect_or_fail("valid_custid", "CustId IS NOT NULL")
@dlt.expect_or_fail("valid_orderid", "OrderId IS NOT NULL")
def ordersummary():
    # Get the joined data with SCD Type 2 columns
    joined_df = dlt.read("customer_order_joined").withColumn(
        "IsActive", lit(True)
    ).withColumn(
        "StartDate", current_timestamp()
    ).withColumn(
        "EndDate", lit(None).cast(TimestampType())
    )
    try:
        existing_df = spark.table(f"{CATALOG}.{SCHEMA}.ordersummary")
    )
    return joined_df
except:
    return joined_df

```

Step 7: Aggregate Customer Spend

The code defines a table, `customeraggregatespend`, which aggregates the customer spend by name and date.

```

@dlt.table(
    name="customeraggregatespend",
    comment="Aggregated customer spend by name and date"
)
def customeraggregatespend():
    ordersummary_df = dlt.read("ordersummary").filter("IsActive = true")
    return ordersummary_df.groupBy("Name", "Date")
        .agg(sum_("TotalAmount").alias("TotalAmount"))

```